



KTU NOTES

The learning companion.

**KTU STUDY MATERIALS | SYLLABUS | LIVE
NOTIFICATIONS | SOLVED QUESTION PAPERS**

Website: www.ktunotes.in

Module 2

Solving problems by searching - problem solving agents - illustration of the problem solving process by agents - searching for solns - Uninformed

Search strategies: BFS, uniform-cost search, DFS, Depth limited search, iterative deepening depth

first search, Informed search strategies: Best-first search

informed search strategy: A* search, Heuristic functions.

Solving problems by searching - problem solving agents

Reflex agent is simple

→ base their actions on

→ A direct mapping from states to actions.

→ But cannot work well in environments

* which this mapping would be too large to store.

* and would take too long to learn.

Hence goal based agent is used.

Problem Solving agent

- Pso is a kind of goal based agent
- It solve problem by
 - * finding sequences of actions, that lead - to desirable state (goals)
- * To solve problem,
 - * The first steps is the goal formulation based on the current situation.

Goal formulation

- The goal is formulated
 - * as it a set of world states, in which the goal is satisfied
- Reaching from Initial state $\Rightarrow$ goal state
 - * Actions are sequenced.
- Actions are the operators
 - * causing transitions between world state.
 - * Actions should be abstract enough at a certain degree. Instead of very detailed
Eg: turn left Vs turn left 30 degree etc.

Problem formulation

The process of deciding

- * what actions and state to consider
- Eg: driving Amman $\rightarrow$ Zarqa

* In b/w States and actions defined.

* States: Some places in amman, amman & Zarga

* Actions :- Turn left, Turn right, go straight, accelerate, brake, etc.

Search

→ Because there are many ways to achieve the same goal.

* Those ways are together expressed as a tree

* Multiple options of unknown value at a point.

* The agent can examine different possible sequences of actions, and choose the best.

* This process of looking for the best sequence is called search

* The best sequence is then a list of actions

* Because there are many ways to achieve the same goal.

* These ways are together expressed as a tree.

* Multiple options of unknown value at a point.

* This process of looking for the best sequence called Search.

* The best sequence is then a list of actions called Solution.

Search algorithm

→ Defined as

* Taking a problem.

* and returns a solution.

→ Once a Solution is found

* The agent follows the solution

↓ and carries out the list of actions - execution phase.

→ Design of an agent

* "formulate", search, execute.

Eg:- Romania map

Well defined problems & Solutions

A problem is defined by 5 components.

- 1) Initial state.
- 2) Actions
- 3) Transition model (or)
- 4) Successor fn
- 5) Goal test
- 7) path cost

→ A problem is defined by 4 components.

→ Initial state

* That agent starts in

→ * The set of possible actions

* Transition model : description of what each action does.

* Successor fn :- Refers to any state reachable from given state by a single action

→ * Initial state ; - actions and transition model define the state space.

* The set of all states reachable from the initial state by any sequence of actions.

→ A path in the state space

* Any sequence of states connected by a sequence of actions.

→ * The goal test

* Applied to the current state to test.

* If the agent is in its goals.

* Sometimes there is an explicit set of possible goal state.

* Sometimes the goal is described by the properties

* Instead of stating explicitly the set of states.

* Example: Chess.

* The agent wins if it can capture the king of the opponent on next move.

* No matter what the opponent does.

→ The path cost function:-

* Assign a numerical cost to each path

* performance measure.

* denoted by g .

* To distinguish the best path from others.

→ Usually the path cost is,

* The sum of the steps cost of the individual actions

→ Together a problem is defined by,

* Initial state

* Actions

* Successor function

* Goal test

* Path cost function.

→ The solⁿ of a problem is then

* A path from the Initial state to state satisfying the goal test.

→ Optimal solⁿ

* A solⁿ with lowest path cost among all solⁿ.

Formulating problem

→ Besides the four components for problem formulation.

→ Abstraction

* The process to take out the irrelevant if

* Leave the most-essential parts to the description of the states

Conclusion

Only the most important parts that are contributing to searching are used

Single state problem formulation

A problem is defined by four items,
Initial state, eg: "at Arad".

Initial state eg:

1. actions or Successor functions $S(x)$ = set of action - state pairs.

eg: $S(\text{Arad}) = (\text{c-Arad} \rightarrow \text{zerind}, \text{zerind})$

2. goal test- can be

* explicit eg. $x = \text{"at Bucharest"}$

* implicit, eg: $\text{checkmate}(x)$

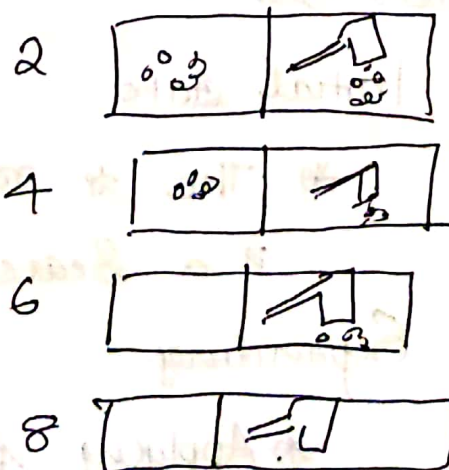
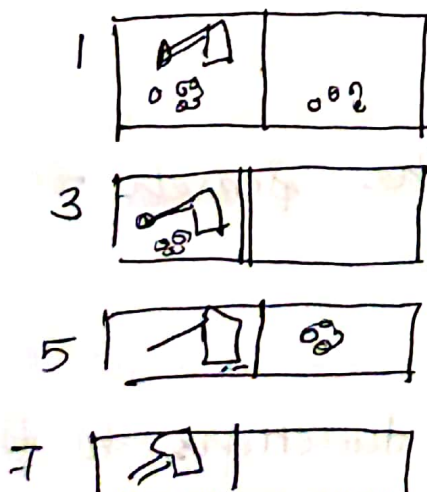
3. path cost (additive)

* eg: sum of distances, number of actions executed etc.

* $c(x, a, y)$ is the step cost, assumed to be > 0

A solution is a sequence of actions leading from the initial state to a goal state.

Toy problem



- No. of state = 8
- Initial state : Any
- No. of actions = 4
 - left, right, suck, no Op
- Goal : clean up all dirt
- Goal state {7, 8}
- Path cost
 - Each step cost 1

Searching for Solutions

- Finding out a solution is done by
 - * Searching through a state space
- All problems are transformed
 - * As a search tree
 - * generated the initial state and successor function.

Search tree :-

Initial state

- * The root of the search tree is a search node

Expanding

- * Applying successor functions to the

Current state

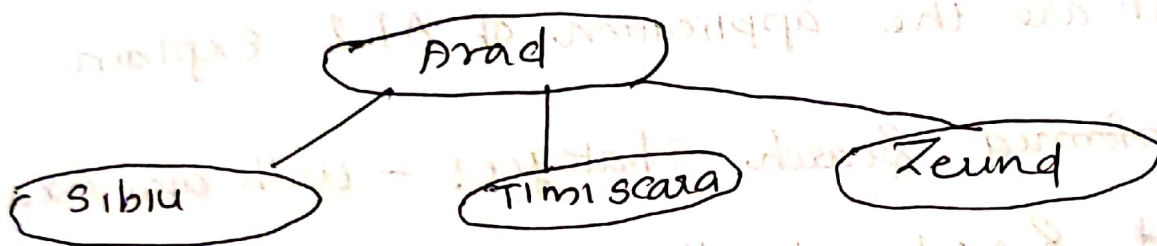
* Thereby generating a new set of states.

Leaf node

* The states having no successors

* Fringe $\Rightarrow$ Set of search nodes that have not been expanded yet.

Tree Search Example



Measuring problem Solving Performance

$\Rightarrow$ For effectiveness of a search algorithm

* We can just consider the total cost.

* The total cost = path cost (g) of the solution found + Search cost.

$\Rightarrow$ * Trade off

* (long time, optimal solution with least g)

* Vs (shorter time, solⁿ with slightly larger path cost. g).

Uninformed Search strategies.

- No additional info about states beyond that presented in the problem defⁿ
- All they can do is generate successors and distinguish a goal state from a non goal state.

BFS

- These algms ignore where they are going until — they find a goal and report success.
 - Is an instance of general graph search algms.
 - In which the shallowest unexpanded node is chosen for expansion
 - FIFO order
 - new nodes
- These algms ignore where they are going until they find a goal and report success.

BFS

- root node expanded first → then all the successors of the root node are expanded next, then their successors. and so on
- all the nodes expanded at a given depth in the search tree before any nodes at the next level are expanded.
- general graph search algms
- FIFO queue for the frontier
- new nodes goes back to the queue, and old nodes which shallower than that the new nodes get expanded first

- The goal test is applied to each node when it is selected for expansion.
- BFS has the shallowest path to every node on the frontier.

Depth first search

- It is an algm. for traversing / searching.
- algm. starts from root node & explores as far as possible along each branch before backtracking.
- DFS algm. starts with initial node of graph & goes deeper & deeper until we find the goal node. (or) node which has no children. Then the algm. backtracks from the dead end towards the most recent node that yet to be completely unexplored.
- For DFS we use ^{the ds} stack.
- process similar to BFS.
- In DFS the edges that lead to unvisited nodes are called discovery edges. The edges that lead to an already visited node are called backtracking edges.
- It is a recursive algm., use idea of backtracking.
- ~~when~~ backtracking is the ; when you are moving forward and there are no more nodes

along the current-path, you move backwards on the same path to find nodes to traverse:

→ This recursive nature of DFS can be implemented using stacks. The basic ideas,

Step 1: Pick a starting node, and push all its adjacent nodes into stack

Step 2: pop a node from stack, to select next node to visit. push all adjacent nodes into a stack.

Step 3: Repeat this process until the stack is empty; ensure that visited nodes are marked. This will prevent you from visiting the same node more than one time. If you do not mark the visited node → then you visited the same node more than once → you may end up in an infinite loop.

Algo

Step 1: ⇒ Set status = 1 (ready state) for each node in G.

Step 2 ⇒ push the starting node A on the stack and set its status = 2 (waiting state)

Step 3 ⇒ Repeat step 4 and 5 until stack is empty.

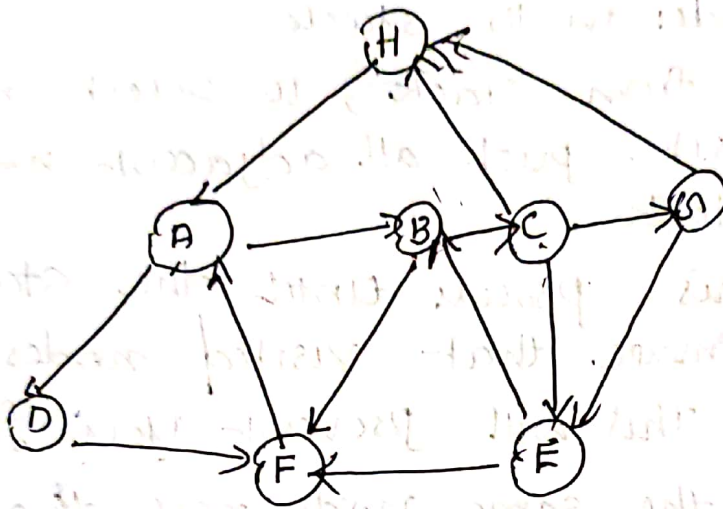
Step 4 ⇒ pop the top node N. process it and set its status = 3 (processed state)

Step 5 ⇒ push on the stack all the neighbours of N that are in the ready state

whose status = 1 and set their status = 2 (waiting state)

[End of loop]

Step 6: Exit



Adjacency list

A : B, D

B : C, F

C : E, G, H

G : E, H

E : B, F

F : A

D : F

H : A

Steps

→ push H on to the stack - Stack 'H'

→ pop the top element of the stack i.e., H, print it and push all the neighbours of H on to stack that are in ready state - print H / Stack: 'A'.

→ pop the top element of the stack i.e., 'A' print it and push all the neighbours of A on to stack that are in ready state print 'A', Stack: B, D

→ pop the top element of the stack 'D', print it and push all the neighbours of D on to stack that are in ready state print 'D', Stack: B, F

- pop the top element 'F', print it and push all the neighbours of 'F' on to stack that are in ready state. print 'F' Stack: B
- pop the top of the stack $\Rightarrow$ B and push the all neighbours of B'. Stack C, F
- pop the top of the stack 'C' and push all neighbours of 'C' on to stack that are in ready state print 'C', Stack = E, G
- pop the top of stack = 'G' - push all neighbours print 'G' Stack $\Rightarrow$ E
- pop the top of the stack 'E', and push all neighbours of 'E' print 'E' Stack:
- hence the stack now become empty and all the nodes of graph have been searched
Sequence $\Rightarrow$ H $\rightarrow$ A $\rightarrow$ D $\rightarrow$ F $\rightarrow$ B $\rightarrow$ C $\rightarrow$ G $\rightarrow$ E $\rightarrow$

- Let d = the depth of search tree = no. of level of the search tree
- $n^d \Rightarrow$ no. of nodes in level i .

- Time complexity $\Rightarrow$ equivalent to the no. of nodes traversed in DFS

$$T(n) = 1 + n^2 + n^3 + n^d = O(n^d)$$

- Space complexity $\Rightarrow$ equivalent to how large can the fringe get, $S(n) = O(n/\text{time})$

- Completeness $\Rightarrow$ DFS is completeness if the search tree is finite, meaning for a given finite search tree, DFS will come up with a soln if it exists.
- Optimality $\Rightarrow$ DFS is not optimal, meaning the no. of steps in searching the soln, (or) the cost spent in searching it is high.

Breadth first Searching

- BFS is an algm for traversing (or) Searching tree (or) graph datastructures. It starts at tree root and explores all of the neighbouring node at the present depth prior to moving on to the nodes at the next depth level.

→ The algm of breadth first search is given below.
The algm starts with examining the node A and all of its neighbours. In the next step - the neighbours of the nearest node of 'A' are explored and process continues in the further steps. The algm explores all neighbours of all nodes and ensures that each node is visited exactly once and no node is visited twice.

Step - Algorithm -

Step 1: SET status = 1 (ready state)
for each node in G.

Step 2: Enqueue the starting node A.
and set its status = 2 (waiting state)

Step 3: Repeat step 4 & 5 until Queue is empty

Step 4: Dequeue a node N. process it and set
its STATUS = 3 (processed state)

Step 5: Enqueue all the neighbours of 'N' that
are in ready state. whose STATUS = 1
(ready state)

and then their status = 2 (waiting)

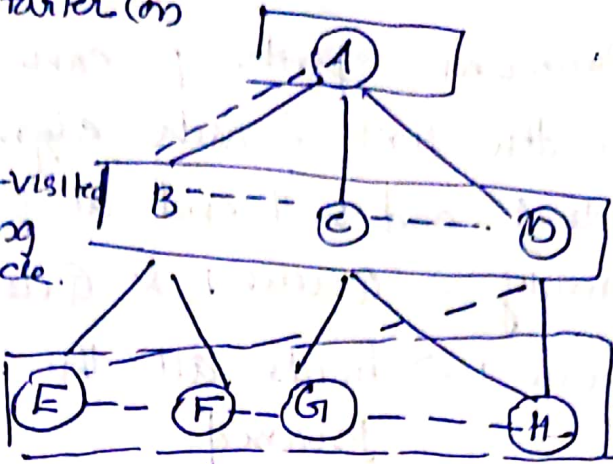
[End of Loop]

Step 6: Exit.

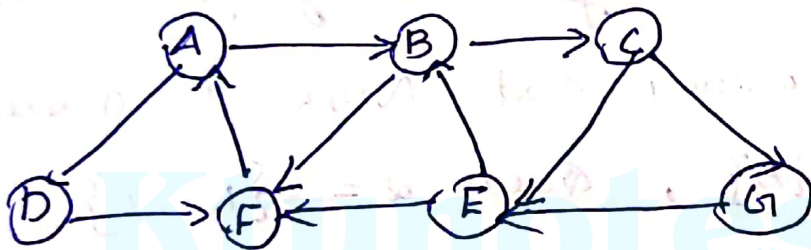
① Mark any node as starter (or initial)

② Explore and traverse un-visited nodes adjacent to starting node.

③ Mark node as completed and move to next adjacent & un-visited nodes.



Example :



Consider the graph G shown in the following image. Calculate the minimum path 'p' from node 'A' to node 'E'. Given that each edge has a length of 1.

Adjacency List-

A : B, D

B : C, F

C : E, G

G : E

E : B, F

F : A

D : F

Solⁿ

Minimum Path 'p' can be found by applying Breadth first search algm. that will begin at node A and will end at E. the algm uses 2 Queue namely - Queue 1 & Queue 2.

Queue 1 $\Rightarrow$ holds all the nodes that are to be processed

Queue 2 $\Rightarrow$ holds all the nodes that are processed and deleted from Queue 1.

Let's start examining the graph from Node

1. Add 'A' to Queue 1 and NULL to Queue 2.

Queue 1 = {A}, Queue 2 = {NULL}

2. Delete the Node 'A' from Queue 1 and Insert all its neighbours. Insert node A to Queue 2

Queue 1 = {B, B} Queue 2 = {A}

3. Delete the node 'B' from Queue 1 and insert all its neighbours. Insert node 'B' to Queue 2.

Queue 1 = {D, C, F} Queue 2 = {A, B}

4. Delete the node 'D' from Queue 1 & Insert all its neighbours. Insert node 'D' to Queue 2

Queue 1 = {C, F} Queue 2 = {A, B, D}

5: Delete the node 'C' from Queue 1 and insert all of its neighbours. Insert the node 'C' to Queue 2

Queue 1 = {E, G} , Queue 2 = {A, B, D, C}

6. Delete F from Queue 1 & insert all of its neighbouring nodes (already been added) no need to add them again). & insert F to Queue 2

Queue 1 = {E, G} Queue 2 = {A, B, D, C, F}

7. Delete 'E' from Queue 1 & insert all of its neighbouring nodes of 'E' (already inserted). & insert 'E' to Queue 2

Queue 1 = {E}, Queue 2 = {A, B, D, C, F, E}

Now back track from E to A using nodes available in Queue 2.

The minimum path will be A → B → C → E

Iterative Deepening Depth first search

→ ^{Queue} BFS → It will find goal node at any way.
It consumes very much memory.

→ ^{Stack} DFS → It consumes very less memory.

→ While still an unintelligent algo. the iterative deepening search combines the +ve elements of BFS and DFS searching to create an algo which is often an improvement over each method individually.

→ Iterative deepening DFS (IDDFS) is an algo that is an important part of an Uninformed search strategy like BFS & DFS.

→ We can define IDDFS as an algo of an ^(combination) amalgam of BFS and DFS.

→ In IDDFS we have found certain limitation in BFS & DFS so we have done hybridization of both the procedure for ~~limiting~~ eliminating the demerits lying in them individually.

→ We do limited DFS up to fixed "limited depth" then we keep on incrementing the depth limit by iterating the procedure unless we have found the goal node or have traversed the whole tree whichever is earlier.

Working Algm of IDDFS

- In the uninformed searching strategy the BFS and DFS have not been so ideal in searching the element in optimum time and space
- The algm only guarantee that the path will be found in exponential time & space.
- So we find a method where we can use the amalgamation of space competence of DFS and optimum solⁿ approach of BFS methods and there we develop a method called iterative deepening using two of them
- There is a maximum depth which define how many levels deep the algm can look for solⁿ.
- A node at the maximum level of depth is treated as terminal, even if it would ordinary have successor node.

Algorithm

- If search "fails" then the maximum level is increased by one and the process repeats. The value for max depth is initially $\Rightarrow 0$.

1. Set SEARCH-DEPTH = 0

2. Consider a depth first search to a depth of

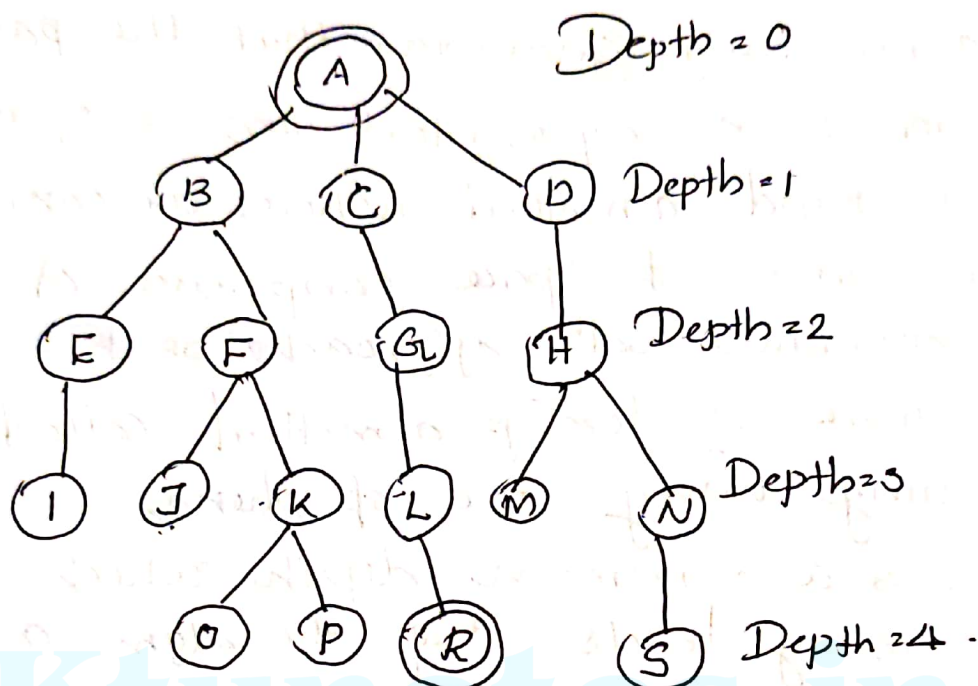
SEARCH-DEPTH .. If a solⁿ found

↓
Th.

3. Otherwise Increment SEARCH_DEPTH

→ This continues until a solution is found.

Example



→ The depth from the figure is 4.

→ In this example we consider the tree as a finite tree, while we can consider the same procedure for the infinite tree as well.

→ We know that the algm of IDDFS we first do DFS till a specified depth and then increase the depth at each loop.

→ The Special Steps forms the part of PLS on depth limited search. The following traversal shows the IDDFS search

The tree can be visited as A B E F C G D H

Depth = $\{0, 1, 2, 3, 4\}$.

Depth limits

Indices

0

A

1

A B C D

2

A B E F C G D H

3

A B E I F J K C G L D H M N

4

A B E I F J K O P C G L R D H
M N S.

UNIFORM - COST SEARCH

- It is used for traversing a weighted tree or graph.
- It comes in to play when a different cost is available for each edge.
- The goal of UCS is to find a path to the goal node which has lowest cumulative cost.
- It expands nodes according to their path cost from the root.
- It can be used to solve any graph/tree where the optimal cost is in demand.
- A UCS algm is implemented by the priority Queue.
- It gives minimum priority to the lowest cumulative cost.

* It is equivalent to BFS if the path cost of all edges is the same.

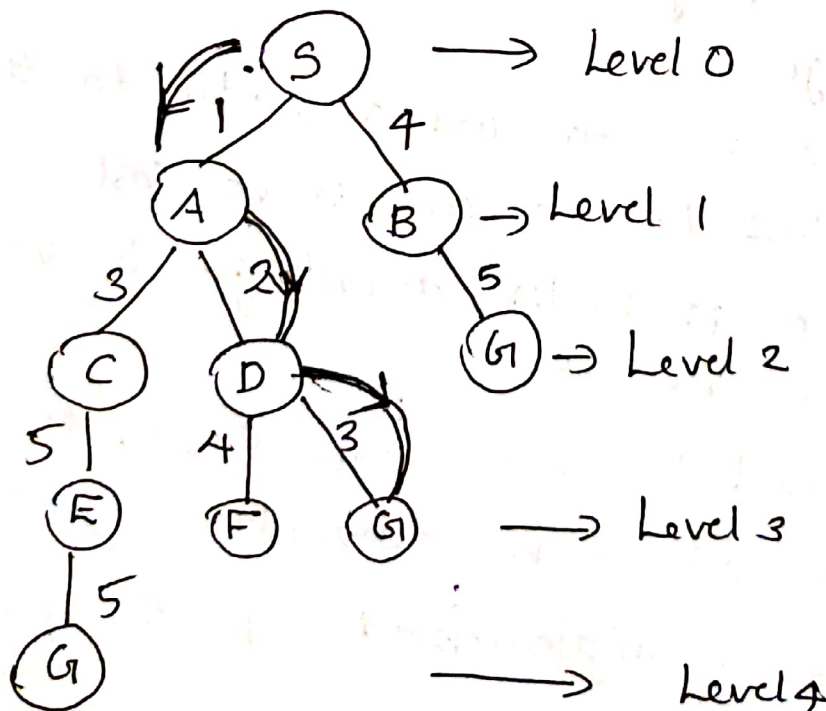
Advantages

→ Uniform Cost Search is Optimal because at every state the path with the least cost is chosen.

Drawbacks

→ It does not care about the no. of steps involve in searching and only concerned about the path cost. Due to which this algm may stuck in infinite loop.

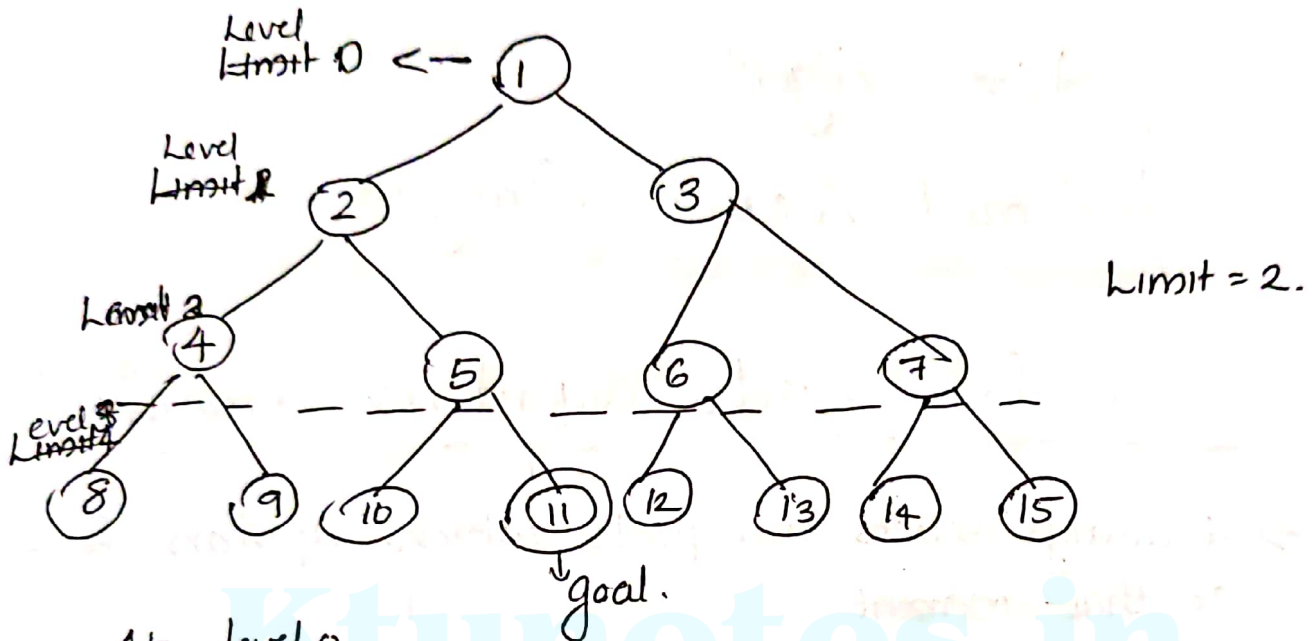
Example



Depth Limited Search - Uninformed search

→ The embarrassing failure of DFS in infinite state space can be alleviated by supplying DFS with a predetermined depth limit 'L'.

→ This approach is called Depth limited search.



At level 0,

node ① will be pick then added their children

2	3
---	---

Level 1 →

pick node 2, their children added

4	5	6	7
---	---	---	---

Level 2

pick 4, their children is on Limit 3 so no select

pick 5, their children is on Limit 3 so no select

then select node ③ → Their childrens add

6	7	12	13
---	---	----	----

No Goal found So it is incomplete. In this problem.

$$\begin{cases} \text{Space Complexity} = O(bl) \rightarrow \text{linear.} \\ \text{Time complexity} = O(b^l) \end{cases}$$

~~Ado~~ Algorithm

Informed Search Strategies

Best first search algorithm (Greedy ^{search} algorithm)

- It always selects the path which appears best at that moment.
- It is a combination of BFS and DFS.
- It uses the heuristic function $h(n) \leq h^*(n)$ ^{Heuristic cost}

$h(n)$ = Heuristic cost

$h^*(n)$ = estimated cost

- The greedy best first search algorithm is implemented by priority Queue.

Algorithm

Step 1: place the starting node n to the open list.

Step 2: If the open list is empty, Stop & return failure.

Step 3 : Remove the node n , from the open list which has the lowest-value $h(n)$, & place it in the closed list.

Step 4 : Expand the node n , and generate the successors of node n .

Step 5 : check each successors of node n and find whether node is a goal node or not.
if any successor node is a goal node, then return success & terminate the search

Step 6 : For each successor node again check for evaluation function $F(n)$ & then check if the node has been either open or closed list. If the node has not been in both list, then add it to Open list.

Step 7 : Return to step 2.

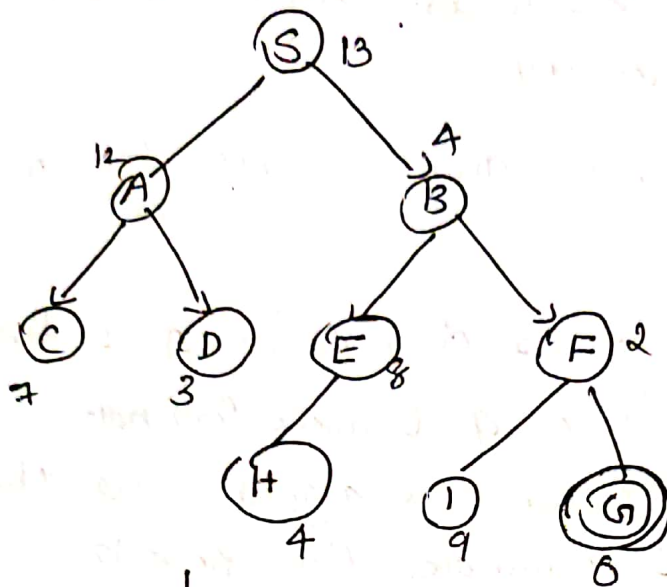
Advantages

- BFS can switch b/w BFS & DFS by gaining the advantages of both the algm
- It is more efficient

Drawbacks

- It can get stuck in a loop as DFS
- This algm is not optimal.

Example :-



node	$h(n)$
S	13
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
G	0

We use 2 list open list (O), closed list (C).

⇒ Initialization

open [A, B] close [S]

⇒ Iteration 1

open [A] close [S, B]

Iteration 2

Open [E, F, A] closed [S, B, ~~B~~]

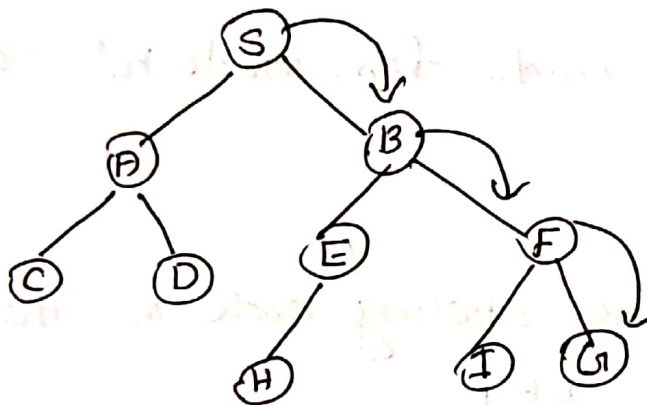
open [E, A] closed [S, B, F]

Iteration 3

Open [E, A, I, G] closed [S, B, F]

Open [E, B, I] closed [S, B, F, G]

The final solⁿ path $S \rightarrow B \rightarrow F \rightarrow G$.



Time complexity = $O(b^n)$

Space Complexity = $O(b^m)$

A* ALGORITHM

- * A* algm finds the shortest path through the search space using the heuristic function.
- * It uses $f(n)$ & cost to reach the node n from the starting stage $g(n)$
- * This algm expands less search tree and provides optimal results faster.
- * It is similar to UCS except that it uses $g(n) + h(n)$ instead of $g(n)$.

* A* Uses Search heuristic as well as the cost to reach the node. Hence we combine both cost as,

$$f(n) = g(n) + h(n) \quad \{ \text{fitness no.} \}$$

$f(n)$ = Estimated cost of cheapest solⁿ.

$g(n)$ = cost to reach node n from start state.

$h(n)$ = cost to reach from node ' n ' to goal node.

Algorithm

Step 1: place the starting node in the Open list

Step 2: check the open list - is empty or not - if the list is empty then return failure

Step 3: Select the node from Open List which has the small value of Evaluation function ($g+h$), if node n is goal node then return success & stop. Otherwise,

Step 4: Expand node ' n ' & generate all of its successors & put ' n ' in the closed list

∴ For each successor ' s ' check whether ' n ' is already in the open & closed list.

* If not then compute Evaluation function for n' and place in to Open list

Step 5: Else if node ' n ' is already in Open & Else closed, then it should be attached to the backpointer which reflects the lowest $g(n')$ value.

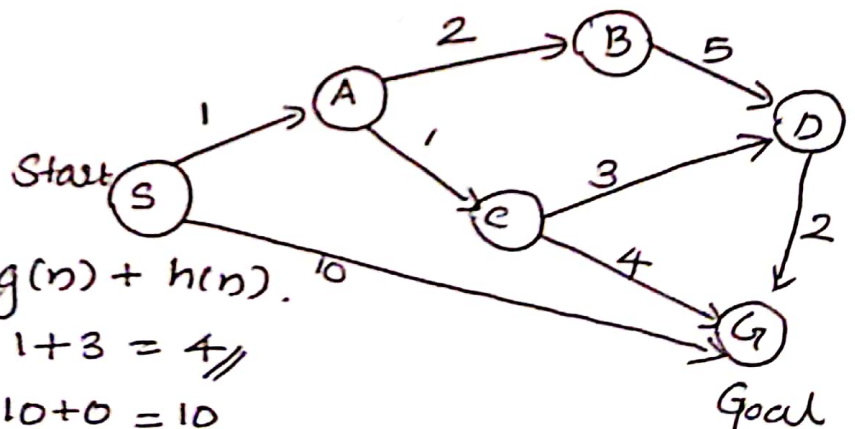
Step 6: Return to step 2.

Advantages

→ it is the best algorithm than other search algm
→

State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Example



$$1) S \rightarrow A \Rightarrow F(n) = g(n) + h(n) \\ = 1 + 3 = 4$$

$$S \rightarrow G \Rightarrow F(n) = 10 + 0 = 10$$

$$2) S \rightarrow A \rightarrow B \Rightarrow F(n) = 3 + 4 = 7$$

$$S \rightarrow A \rightarrow C \Rightarrow F(n) = 2 + 2 = 4$$

$$3) S \rightarrow A \rightarrow C \rightarrow D \Rightarrow F(n) = 5 + 6 = 11$$